

EX.NO:5 ADT QUEUE USING EXCEPTION HANDLING

PROGRAM:

```
import java.io.*;
interface MyQueue
{
    void enqueue();
    void dequeue();
    void display();
}
class CircularQueue implements MyQueue
{
    final int SIZE = 5;
    int queue[] = new int[SIZE];
    int front = -1;
    int rear = -1;
    public void enqueue()
    {
        try
        {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            if((front == 0 && rear == SIZE - 1) || (front == rear + 1))
            {
                System.out.println("Queue Overflow");
                return;
            }
            System.out.print("Enter the element: ");
            int item = Integer.parseInt(br.readLine());
            if(front == -1)
            {
                front = 0;
                rear = 0;
            }
            else if(rear == SIZE - 1)
            {
                rear = 0;
            }
            else
            {
                rear++;
            }
            queue[rear] = item;
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
    public void dequeue()
    {
        if(front == -1)
        {
```

```

        System.out.println("Queue Underflow");
        return;
    }
    System.out.println("Deleted Element: " + queue[front]);
    if(front == rear)
    {
        front = -1;
        rear = -1;
    }
    else if(front == SIZE - 1)
    {
        front = 0;
    }
    else
    {
        front++;
    }
}
public void display()
{
    if(front == -1)
    {
        System.out.println("Queue is Empty");
        return;
    }
    System.out.println("Queue Elements:");

    if(front <= rear)
    {
        for(int i = front; i <= rear; i++)
        {
            System.out.print(queue[i] + " ");
        }
    }
    else
    {
        for(int i = front; i < SIZE; i++)
        {
            System.out.print(queue[i] + " ");
        }
        for(int i = 0; i <= rear; i++)
        {
            System.out.print(queue[i] + " ");
        }
    }
    System.out.println();
}
}

```

```

public class Main
{
    public static void main(String args[]) throws Exception
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        CircularQueue q = new CircularQueue();
        int ch;
        do
        {
            System.out.println("\n*** CIRCULAR QUEUE ***");
            System.out.println("1. Enqueue");
            System.out.println("2. Dequeue");
            System.out.println("3. Display");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            ch = Integer.parseInt(br.readLine());
            switch(ch)
            {
                case 1:
                    q.enqueue();
                    break;
                case 2:
                    q.dequeue();
                    break;
                case 3:
                    q.display();
                    break;
                case 4:
                    System.out.println("Program Ended");
                    break;
                default:
                    System.out.println("Invalid Choice");
            }
        } while(ch != 4);
    }
}

```

OUTPUT:

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 10

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 20

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 30

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 40

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 50

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue Elements:

10 20 30 40 50

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

Deleted Element: 10

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue Elements:

20 30 40 50

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter the element: 60

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue Elements:

20 30 40 50 60

*** CIRCULAR QUEUE ***

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 4

Program Ended

**EX.NO:6 DEMONSTRATE THE IMMUTABLE NATURE OF WRAPPER CLASSES
(INTEGER AND DOUBLE)**

PROGRAM:

```
class WrapperImmutableDemo
{
    public static void main(String args[])
    {
        // Integer Wrapper
        Integer i = Integer.valueOf(100);

        System.out.println("Before Modification");
        System.out.println("Integer Value : " + i);
        System.out.println("Hash Code    : " + System.identityHashCode(i));

        // Modification
        i = i + 50;

        System.out.println("\nAfter Modification");
        System.out.println("Integer Value : " + i);
        System.out.println("Hash Code    : " + System.identityHashCode(i));

        // Double Wrapper
        Double d = Double.valueOf(25.5);

        System.out.println("\nBefore Modification");
        System.out.println("Double Value : " + d);
        System.out.println("Hash Code    : " + System.identityHashCode(d));

        // Modification
        d = d * 2;

        System.out.println("\nAfter Modification");
        System.out.println("Double Value : " + d);
        System.out.println("Hash Code    : " + System.identityHashCode(d));
    }
}
```

OUTPUT:

Before Modification

Integer Value : 100

Hash Code : 1510467688

After Modification

Integer Value : 150

Hash Code : 1995265320

Before Modification

Double Value : 25.5

Hash Code : 746292446

After Modification

Double Value : 51.0

Hash Code : 1072591677